

AI EMAIL COPILOT

RAG-Powered Custom Email Generator

Unique Team ID:	IBMBH00696
Team Name:	GEN CREW
Team Leader:	Sparsh Gupta
Team Size:	5 Members
College Name:	AJAY KUMAR GARG ENGINEERING COLLEGE

Date:

1. Executive Summary / Abstract

The AI Email Copilot is a production-grade enterprise productivity application designed to automate and optimize custom, context-aware email generation. While generic Large Language Model (LLM) templates often result in tone inconsistencies and layout errors, our solution couples Retrieval-Augmented Generation (RAG) with Google Gemini to guarantee compliance with high-quality, pre-vetted corporate templates.

The architecture is decoupled into a robust REST API hosted on AWS Elastic Beanstalk (Amazon Linux 2023) and an interactive front-end client hosted on Streamlit Cloud. By migrating from local, resource-heavy PyTorch-based vectorization libraries to cloud-hosted Gemini embeddings, the server footprint is minimized, achieving absolute reliability on AWS free-tier hosting.

2. Introduction & Objectives

Modern corporate communication requires highly structured, contextually relevant messages tailored to tone, length, and specific purposes. The core objectives of this project include:

- **Consistency:** Ensuring generated outputs adhere to organizational structures.
- **Flexibility:** Allowing multiple tone variants (e.g., professional, friendly, persuasive) and lengths.
- **Cloud Independence:** Structuring a clean REST Backend API accessible from any frontend client.
- **Efficiency:** Designing the RAG pipeline to prevent memory leaks and out-of-memory (OOM) failures.

3. Technical Specifications & Stack

The system stack has been selected to minimize memory overhead while maintaining high similarity search performance:

- **Programming Language:** Python 3.11
- **RAG Framework:** LangChain-Core, LangChain-Google-GenAI
- **Vector Database:** FAISS (Facebook AI Similarity Search)
- **Embeddings Model:** Google gemini-embedding-2 (3072-D)
- **Generation LLM:** Google Gemini 3.5 Flash
- **Backend API Hosting:** AWS Elastic Beanstalk (t3.micro)
- **Frontend App Hosting:** Streamlit Cloud

4. Directory Structure

```
AI_Email_Generator_Copilot/  
|-- api.py                # FastAPI REST Server
```

```
|-- streamlit_app.py      # Streamlit Frontend Client
|-- Procfile             # AWS EB Startup Instructions
|-- requirements.txt      # Streamlit Dependencies
|-- requirements-api.txt  # AWS EB Dependencies
|-- .ebextensions/
|   +-- python.config     # Python platform parameters
|-- rag/
|   |-- build_vectorstore.py # Generates vector db index
|   +-- retriever.py       # Handles FAISS similarity search
+-- faiss_index/          # Saved vector database files
```

5. RAG Pipeline Architecture

The retrieval pipeline queries custom email templates saved in raw text files. The data flows sequentially:

1. Semantic Query generation: User inputs (e.g., purpose, recipient, key points) are concatenated to build a dense semantic search string.
2. Vectorization: The search string is converted into a 3072-dimensional vector using the google gemini-embedding-2 model.
3. Local Search: The query vector is scanned against a pre-built local FAISS index, extracting top-K contextually matching templates.
4. Prompt Compilation: The retrieved templates are injected into the Prompt Template as few-shot context.
5. Generation: Gemini 3.5 Flash processes the unified prompt to output the final formatted email.

6. Key Implementations: api.py

```
@app.post('/generate-email')
def generate_email(request: EmailRequest):
```

```
# Step 1: Retrieve matching templates from vector database
templates = retrieve_context(rag_query)
# Step 2: Inject retrieved templates into LLM prompt
prompt = build_email_prompt(..., context=templates)
# Step 3: Run Gemini model generation
response = model.generate_content(prompt)
return {'success': True, 'data': {'email': response.text}}
```

7. Major Implementation Challenges & Fixes

Issue A: Memory Exhaustion (OOM) on Free-Tier Instances

Originally, the FAISS database was backed by local Hugging Face all-MiniLM-L6-v2 embeddings via PyTorch. Because PyTorch and tokenizers require over 1.2 GB of RAM during startup, the t3.micro instance (1GB memory limit) regularly crashed. To fix this, local vectorization libraries were removed and replaced with Google's hosted gemini-embedding-2 API, resulting in a lightweight, stable, and serverless vector search process.

Issue B: Windows ZIP Formatting Extraction Errors

Deploying standard ZIP archives compiled using Windows utility libraries resulted in unzip failures on the AWS Amazon Linux 2023 platform because paths were formatted with backslashes (\). We bypassed this by generating the deployment archive using git archive, forcing Linux-compatible forward-slash (/) paths.

Issue C: Procfile Parsing Interruptions

Windows system text editors often write carriage returns (CRLF) in Procfiles, causing process managers on Elastic Beanstalk to throw errors. We resolved this by explicitly stripping windows characters and formatting the file with exactly one line terminated with a Unix Linefeed (LF).

8. Verification & Production Test Case

The live REST API hosted on AWS Elastic Beanstalk was tested with the following HTTP POST payload:

```
{
  'sender_name': 'Sparsh',
  'recipient_name': 'Harsh',
  'company_name': 'IBM',
  'purpose': 'follow up',
  'tone': 'professional',
  'length': 'short',
  'key_points': ['Hi', 'Checking in on the project status']
}
```

The server processed the request and successfully returned code 200 with the email:

Subject: Follow-Up: Project Status

Dear Harsh,

I hope you are having a productive week. I am writing to check in on our project's status and see if there are any updates from your side. Please let me know if you need any additional information or support from my end to help move things forward.

Best regards,
Sparsh

9. Conclusion

The project successfully implements a modern, memory-efficient, and enterprise-grade RAG pipeline. By hosting the core API server on AWS Elastic Beanstalk and the interactive interface on Streamlit Cloud, we have delivered a reliable, decoupled cloud application. The system performs under 100ms vector searches and handles high-quality structured drafts at zero cost, making it a highly scalable template generator tool.